

Aula 2 - Regressão Logística

João Florindo

Instituto de Matemática, Estatística e Computação Científica
Universidade Estadual de Campinas - Brasil
florindo@unicamp.br

Outline

- 1 Classificação
- 2 Regressão Logística
- 3 Fronteira de Decisão
- 4 Função de Custo
- 5 Gradiente Descendente

- CLASSIFICAÇÃO: Saída discreta.

- Email é spam ou não
- Transação é fraudulenta ou não
- Tumor benigno ou maligno

- Esses são exemplos de classificação **binária** (2 classes)

- CLASSIFICAÇÃO: Saída discreta.
 - Email é spam ou não
 - Transação é fraudulenta ou não
 - Tumor benigno ou maligno

- Esses são exemplos de classificação **binária** (2 classes)

- CLASSIFICAÇÃO: Saída discreta.
 - Email é spam ou não
 - Transação é fraudulenta ou não
 - Tumor benigno ou maligno

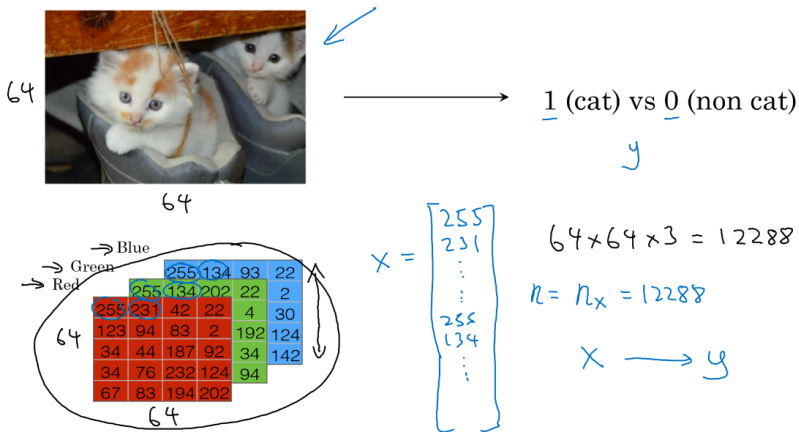
- Esses são exemplos de classificação **binária** (2 classes)

- CLASSIFICAÇÃO: Saída discreta.
 - Email é spam ou não
 - Transação é fraudulenta ou não
 - Tumor benigno ou maligno

- Esses são exemplos de classificação **binária** (2 classes)

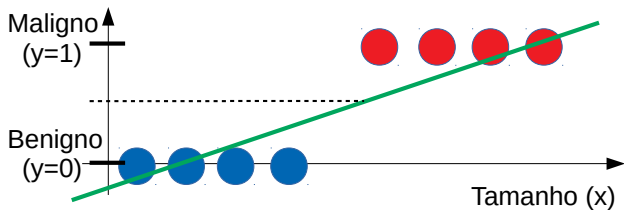
- CLASSIFICAÇÃO: Saída discreta.

- Email é spam ou não
- Transação é fraudulenta ou não
- Tumor benigno ou maligno



- Esses são exemplos de classificação **binária** (2 classes)

Primeira ideia: usar regressão linear



PROBLEMAS:

- Um exemplo de treinamento a mais pode mudar completamente a reta
- Valores de saída fora de $[0, 1]$
- SOLUÇÃO: **Regressão Logística**

PROBLEMAS:

- Um exemplo de treinamento a mais pode mudar completamente a reta
- Valores de saída fora de $[0, 1]$
- SOLUÇÃO: Regressão Logística

PROBLEMAS:

- Um exemplo de treinamento a mais pode mudar completamente a reta
- Valores de saída fora de $[0, 1]$
- SOLUÇÃO: **Regressão Logística**

Outline

- 1 Classificação
- 2 Regressão Logística
- 3 Fronteira de Decisão
- 4 Função de Custo
- 5 Gradiente Descendente

Regressão Logística

Dada a entrada $x \in \mathbb{R}^{n_x}$, obtemos a saída

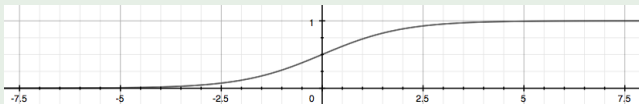
$$\hat{y} = \sigma(\mathbf{w}^T x + b),$$

a qual $\hat{y}$ depende de dois parâmetros $\mathbf{w} \in \mathbb{R}^{n_x}$ e $b \in \mathbb{R}$. $\sigma(z)$ é a função sigmoide (logística):

$$\sigma(z) = \frac{1}{1 + e^{-z}},$$

ou seja:

$$\hat{y}_{\mathbf{w},b}(x) = \frac{1}{1 + e^{-(\mathbf{w}^T x + b)}}.$$



- NOTA: Não usamos mais a notação

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \vdots \\ \theta_{n_x} \end{bmatrix}, \quad x_0 = 1$$

$$\hat{y} = \sigma(\theta^T x)$$

- O uso de $\mathbf{w} \in \mathbb{R}^{n_x}$ e $b \in \mathbb{R}$ é mais adequado em redes neurais

Interpretação

Note que $0 \leq \hat{y} \leq 1$ e o valor de $\hat{y}$ é interpretado como a probabilidade

$$\hat{y}_{\mathbf{w},b}(x) = P(y = 1|x).$$

Lê-se: “probabilidade de $y = 1$, dado x , parametrizado por $\mathbf{w}$ e b ”.

Exemplo:

x = tamanho do tumor

e

$$\hat{y}_{\mathbf{w},b}(x) = 0.7$$

⇒ Probabilidade de o tumor ser maligno ($y = 1$) é 70%.

- Como $\hat{y} \in \{0, 1\}$:

$$P(y = 0|x; \mathbf{w}, b) = 1 - P(y = 1|x; \mathbf{w}, b).$$

Interpretação

Note que $0 \leq \hat{y} \leq 1$ e o valor de $\hat{y}$ é interpretado como a probabilidade

$$\hat{y}_{\mathbf{w},b}(x) = P(y = 1|x).$$

Lê-se: “probabilidade de $y = 1$, dado x , parametrizado por $\mathbf{w}$ e b ”.

Exemplo:

x = tamanho do tumor

e

$$\hat{y}_{\mathbf{w},b}(x) = 0.7$$

⇒ Probabilidade de o tumor ser maligno ($y = 1$) é 70%.

- Como $\hat{y} \in \{0, 1\}$:

$$P(y = 0|x; \mathbf{w}, b) = 1 - P(y = 1|x; \mathbf{w}, b).$$

Outline

- 1 Classificação
- 2 Regressão Logística
- 3 Fronteira de Decisão**
- 4 Função de Custo
- 5 Gradiente Descendente

Fronteira de Decisão

- Uma **fronteira de decisão** separa os exemplos com $y = 1$ dos com $y = 0$
- Definida por $\hat{y}_{\mathbf{w},b}(x)$ e um limiar t : $y = 1$ se $\hat{y}_{\mathbf{w},b}(x) \geq t$ e $y = 0$ caso contrário
- Fronteira de decisão dada por $\hat{y}_{\mathbf{w},b}(x) = t$
- Na regressão logística, se por exemplo $t = 0.5$, temos $y = 1$ se $\sigma(z) \geq 0.5$, ou ainda:

$$y = \begin{cases} 1 & \text{se } \mathbf{w}^T \mathbf{x} + b \geq 0 \\ 0 & \text{se } \mathbf{w}^T \mathbf{x} + b < 0. \end{cases}$$

Fronteira de Decisão

- Uma **fronteira de decisão** separa os exemplos com $y = 1$ dos com $y = 0$
- Definida por $\hat{y}_{\mathbf{w},b}(x)$ e um limiar t : $y = 1$ se $\hat{y}_{\mathbf{w},b}(x) \geq t$ e $y = 0$ caso contrário
- Fronteira de decisão dada por $\hat{y}_{\mathbf{w},b}(x) = t$
- Na regressão logística, se por exemplo $t = 0.5$, temos $y = 1$ se $\sigma(z) \geq 0.5$, ou ainda:

$$y = \begin{cases} 1 & \text{se } \mathbf{w}^T \mathbf{x} + b \geq 0 \\ 0 & \text{se } \mathbf{w}^T \mathbf{x} + b < 0. \end{cases}$$

Fronteira de Decisão

- Uma **fronteira de decisão** separa os exemplos com $y = 1$ dos com $y = 0$
- Definida por $\hat{y}_{\mathbf{w},b}(x)$ e um limiar t : $y = 1$ se $\hat{y}_{\mathbf{w},b}(x) \geq t$ e $y = 0$ caso contrário
- Fronteira de decisão dada por $\hat{y}_{\mathbf{w},b}(x) = t$
- Na regressão logística, se por exemplo $t = 0.5$, temos $y = 1$ se $\sigma(z) \geq 0.5$, ou ainda:

$$y = \begin{cases} 1 & \text{se } \mathbf{w}^T \mathbf{x} + b \geq 0 \\ 0 & \text{se } \mathbf{w}^T \mathbf{x} + b < 0. \end{cases}$$

- Uma **fronteira de decisão** separa os exemplos com $y = 1$ dos com $y = 0$
- Definida por $\hat{y}_{\mathbf{w},b}(x)$ e um limiar t : $y = 1$ se $\hat{y}_{\mathbf{w},b}(x) \geq t$ e $y = 0$ caso contrário
- Fronteira de decisão dada por $\hat{y}_{\mathbf{w},b}(x) = t$
- Na regressão logística, se por exemplo $t = 0.5$, temos $y = 1$ se $\sigma(z) \geq 0.5$, ou ainda:

$$y = \begin{cases} 1 & \text{se } \mathbf{w}^T \mathbf{x} + b \geq 0 \\ 0 & \text{se } \mathbf{w}^T \mathbf{x} + b < 0. \end{cases}$$

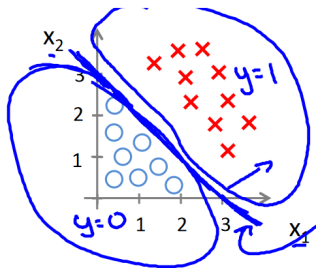
Exemplo

$$\hat{y}_{\mathbf{w},b}(x) = \sigma(w_1x_1 + w_2x_2 + b)$$

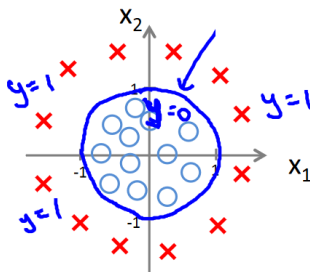
dado que

$$\mathbf{w} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \text{ e } b = -3,$$

$y = 1$ se $x_1 + x_2 - 3 \geq 0$.



E se uma reta não separar os exemplos?



$$\hat{y}_{\mathbf{w},b}(x) = \sigma(w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_2^2 + b),$$

com

$$\mathbf{w} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \end{bmatrix} \text{ e } b = -1,$$

de modo que $y = 1$ se $x_1^2 + x_2^2 - 1 \geq 0$.

Outline

- 1 Classificação
- 2 Regressão Logística
- 3 Fronteira de Decisão
- 4 Função de Custo**
- 5 Gradiente Descendente

- Dados os exemplos de treinamento $\{(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})\}$, encontrar $\mathbf{w}$ e b tais que

$$\hat{y}^{(i)} \approx y^{(i)}.$$

- Definimos uma *loss function* $\mathcal{L}(\hat{y}^{(i)}, y^{(i)})$, que mede a discrepância entre a predição $\hat{y}^{(i)}$ e o valor desejado $y^{(i)}$ para cada exemplo
- Uma primeira ideia seria:

$$\mathcal{L}(\hat{y}^{(i)}, y^{(i)}) = (\hat{y}^{(i)} - y^{(i)})^2,$$

como usado na regressão linear. Porém, esta função não é convexa (mínimos locais)

- Define-então

$$\mathcal{L}(\hat{y}^{(i)}, y^{(i)}) = -(y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}))$$

- Dados os exemplos de treinamento $\{(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})\}$, encontrar $\mathbf{w}$ e b tais que

$$\hat{y}^{(i)} \approx y^{(i)}.$$

- Definimos uma *loss function* $\mathcal{L}(\hat{y}^{(i)}, y^{(i)})$, que mede a discrepância entre a predição $\hat{y}^{(i)}$ e o valor desejado $y^{(i)}$ para cada exemplo

- Uma primeira ideia seria:

$$\mathcal{L}(\hat{y}^{(i)}, y^{(i)}) = (\hat{y}^{(i)} - y^{(i)})^2,$$

como usado na regressão linear. Porém, esta função não é convexa (mínimos locais)

- Define-então

$$\mathcal{L}(\hat{y}^{(i)}, y^{(i)}) = -(y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}))$$

- Dados os exemplos de treinamento $\{(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})\}$, encontrar $\mathbf{w}$ e b tais que

$$\hat{y}^{(i)} \approx y^{(i)}.$$

- Definimos uma *loss function* $\mathcal{L}(\hat{y}^{(i)}, y^{(i)})$, que mede a discrepância entre a predição $\hat{y}^{(i)}$ e o valor desejado $y^{(i)}$ para cada exemplo
- Uma primeira ideia seria:

$$\mathcal{L}(\hat{y}^{(i)}, y^{(i)}) = (\hat{y}^{(i)} - y^{(i)})^2,$$

como usado na regressão linear. Porém, esta função não é convexa (mínimos locais)

- Define-então

$$\mathcal{L}(\hat{y}^{(i)}, y^{(i)}) = -(y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}))$$

- Dados os exemplos de treinamento $\{(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})\}$, encontrar $\mathbf{w}$ e b tais que

$$\hat{y}^{(i)} \approx y^{(i)}.$$

- Definimos uma *loss function* $\mathcal{L}(\hat{y}^{(i)}, y^{(i)})$, que mede a discrepância entre a predição $\hat{y}^{(i)}$ e o valor desejado $y^{(i)}$ para cada exemplo
- Uma primeira ideia seria:

$$\mathcal{L}(\hat{y}^{(i)}, y^{(i)}) = (\hat{y}^{(i)} - y^{(i)})^2,$$

como usado na regressão linear. Porém, esta função não é convexa (mínimos locais)

- Define-então

$$\mathcal{L}(\hat{y}^{(i)}, y^{(i)}) = -(y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}))$$

- Note que:

- Se $y = 1$: $\mathcal{L}(\hat{y}, y) = -\log(\hat{y})$ e o algoritmo maximiza $\hat{y}$ (mais próximo de 1)
- Se $y = 0$: $\mathcal{L}(\hat{y}, y) = -\log(1 - \hat{y})$ e o algoritmo minimiza $\hat{y}$ (mais próximo de 0)

- A função de custo é a média da *loss* sobre todo o treinamento:

$$J(\mathbf{w}, b) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right].$$

- Note que:

- Se $y = 1$: $\mathcal{L}(\hat{y}, y) = -\log(\hat{y})$ e o algoritmo maximiza $\hat{y}$ (mais próximo de 1)
- Se $y = 0$: $\mathcal{L}(\hat{y}, y) = -\log(1 - \hat{y})$ e o algoritmo minimiza $\hat{y}$ (mais próximo de 0)

- A função de custo é a média da *loss* sobre todo o treinamento:

$$J(\mathbf{w}, b) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right].$$

- Note que:

- Se $y = 1$: $\mathcal{L}(\hat{y}, y) = -\log(\hat{y})$ e o algoritmo maximiza $\hat{y}$ (mais próximo de 1)
- Se $y = 0$: $\mathcal{L}(\hat{y}, y) = -\log(1 - \hat{y})$ e o algoritmo minimiza $\hat{y}$ (mais próximo de 0)

- A função de custo é a média da *loss* sobre todo o treinamento:

$$J(\mathbf{w}, b) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right].$$

- Note que:

- Se $y = 1$: $\mathcal{L}(\hat{y}, y) = -\log(\hat{y})$ e o algoritmo maximiza $\hat{y}$ (mais próximo de 1)
- Se $y = 0$: $\mathcal{L}(\hat{y}, y) = -\log(1 - \hat{y})$ e o algoritmo minimiza $\hat{y}$ (mais próximo de 0)

- A função de custo é a média da *loss* sobre todo o treinamento:

$$J(\mathbf{w}, b) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right].$$

Temos:

$$\hat{y} = \sigma(\mathbf{w}^T x + b) \text{ com } \sigma(z) = \frac{1}{1 + e^{-z}}$$

Interpretação:

$$\hat{y} = p(y = 1|x)$$

Assim temos:

$$\text{Se } y = 1: \quad p(y|x) = \hat{y}$$

$$\text{Se } y = 0: \quad p(y|x) = 1 - \hat{y}$$

Podemos juntar em uma única equação:

$$p(y|x) = \hat{y}^y (1 - \hat{y})^{1-y}$$

Temos:

$$\hat{y} = \sigma(\mathbf{w}^T x + b) \text{ com } \sigma(z) = \frac{1}{1 + e^{-z}}$$

Interpretação:

$$\hat{y} = p(y = 1|x)$$

Assim temos:

$$\text{Se } y = 1: \quad p(y|x) = \hat{y}$$

$$\text{Se } y = 0: \quad p(y|x) = 1 - \hat{y}$$

Podemos juntar em uma única equação:

$$p(y|x) = \hat{y}^y (1 - \hat{y})^{1-y}$$

Temos:

$$\hat{y} = \sigma(\mathbf{w}^T x + b) \text{ com } \sigma(z) = \frac{1}{1 + e^{-z}}$$

Interpretação:

$$\hat{y} = p(y = 1|x)$$

Assim temos:

$$\text{Se } y = 1: \quad p(y|x) = \hat{y}$$

$$\text{Se } y = 0: \quad p(y|x) = 1 - \hat{y}$$

Podemos juntar em uma única equação:

$$p(y|x) = \hat{y}^y (1 - \hat{y})^{1-y}$$

Objetivo é maximizar $p(y|x)$ (*maximum likelihood estimation*)

Aplicamos log para facilitar a derivada:

$$\log p(y|x) = y \log \hat{y} + (1 - y) \log(1 - \hat{y})$$

A *loss* porém deve ser minimizada, não maximizada - por isso o sinal de menos:

$$\mathcal{L}(\hat{y}, y) = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y}))$$

Objetivo é maximizar $p(y|x)$ (*maximum likelihood estimation*)

Aplicamos log para facilitar a derivada:

$$\log p(y|x) = y \log \hat{y} + (1 - y) \log(1 - \hat{y})$$

A *loss* porém deve ser minimizada, não maximizada - por isso o sinal de menos:

$$\mathcal{L}(\hat{y}, y) = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y}))$$

Objetivo é maximizar $p(y|x)$ (*maximum likelihood estimation*)

Aplicamos log para facilitar a derivada:

$$\log p(y|x) = y \log \hat{y} + (1 - y) \log(1 - \hat{y})$$

A *loss* porém deve ser minimizada, não maximizada - por isso o sinal de menos:

$$\mathcal{L}(\hat{y}, y) = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y}))$$

Assumimos amostragem i.i.d.:

$$\begin{aligned} p(\text{rótulos no treino}) &= \log \prod_{i=1}^m p(y^{(i)} | x^{(i)}) \\ &= \sum_{i=1}^m \log p(y^{(i)} | x^{(i)}) \\ &= \sum_{i=1}^m y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \end{aligned}$$

Tirando a média e invertendo sinal (minimizar):

$$J(\mathbf{w}, b) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right].$$

Assumimos amostragem i.i.d.:

$$\begin{aligned} p(\text{rótulos no treino}) &= \log \prod_{i=1}^m p(y^{(i)} | x^{(i)}) \\ &= \sum_{i=1}^m \log p(y^{(i)} | x^{(i)}) \\ &= \sum_{i=1}^m y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \end{aligned}$$

Tirando a média e invertendo sinal (minimizar):

$$J(\mathbf{w}, b) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right].$$

Outline

- 1 Classificação
- 2 Regressão Logística
- 3 Fronteira de Decisão
- 4 Função de Custo
- 5 Gradiente Descendente

Gradiente Descendente

- Obter $\mathbf{w}$ e b que minimizam $J(\mathbf{w}, b)$
- Pode inicializar com zero (J é convexa)

```
Repita{  
     $w := w - \alpha \frac{dJ}{dw}$   
}
```

Gradiente Descendente

- Obter $\mathbf{w}$ e b que minimizam $J(\mathbf{w}, b)$
- Pode inicializar com zero (J é convexa)

```
Repita{  
     $w := w - \alpha \frac{dJ}{dw}$   
}
```

Gradiente Descendente

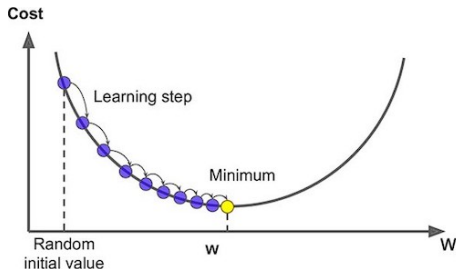
- Obter $\mathbf{w}$ e b que minimizam $J(\mathbf{w}, b)$
- Pode inicializar com zero (J é convexa)

```
Repita{  
     $w := w - \alpha \frac{dJ}{dw}$   
}
```

Gradiente Descendente

- Obter $\mathbf{w}$ e b que minimizam $J(\mathbf{w}, b)$
- Pode inicializar com zero (J é convexa)

```
Repita{  
     $w := w - \alpha \frac{dJ}{dw}$   
}
```



- Seja a expressão $e = (a + b) * (b + 1)$.

- Pode ser quebrada em

$$c = a + b$$

$$d = b + 1$$

$$e = c * d$$

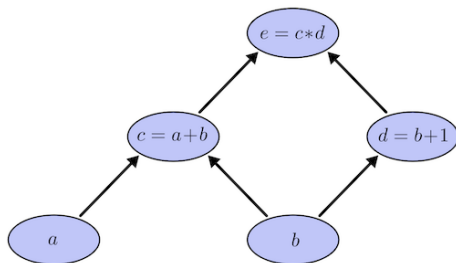
Grafo de Computação

- Seja a expressão $e = (a + b) * (b + 1)$.
- Pode ser quebrada em

$$c = a + b$$

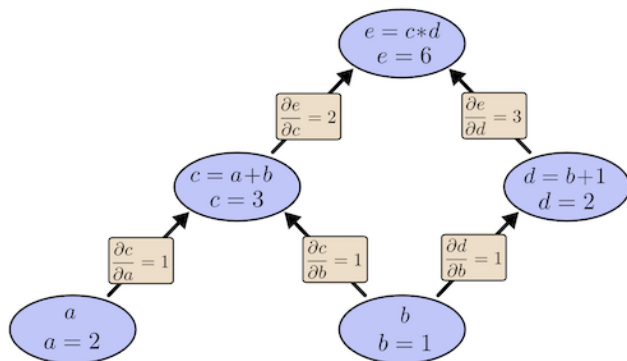
$$d = b + 1$$

$$e = c * d$$



Grafo de Computação - Derivada

- Derivada nas arestas.
- EX.: Como uma pequena alteração em a afeta c ?

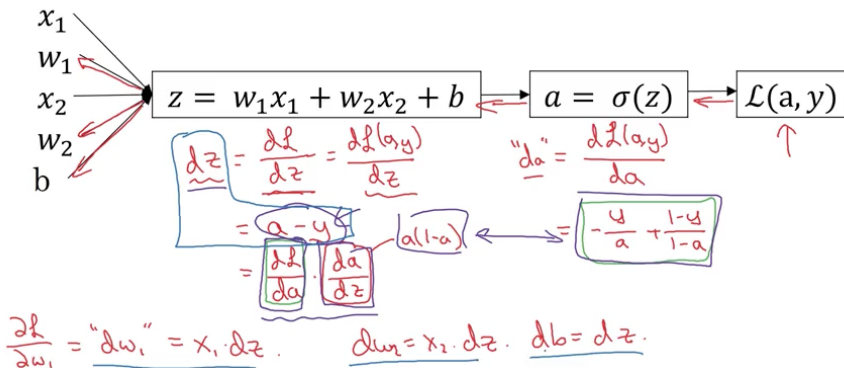


Grafo de Computação - Regressão Logística

$$z = w^T x + b$$

$$\hat{y} = a = \sigma(z)$$

$$\mathcal{L}(a, y) = -(y \log(a) + (1 - y) \log(1 - a))$$



Gradiente Descendente com m Exemplos

- A derivada neste caso também é a média das derivadas

```
j=0
dw=np.zeros(n)
db=0

for i in range(m):
    z[i] = w.transpose() * x[i] + b
    a[i] = sigmoid(z[i])
    J = J + (-[y[i]*log(a[i])+(1-y[i])*log(1-a[i])])
    dz[i] = a[i] - y[i]

    # inner loop for n features, later will be optimized by vectorization
    for j in range(n):
        dw[j] = dw[j] + x[i][j] * dz[i]

    db = db + dz[i]

j = j / m
dw = dw / m
db = db / m
```

- Atualização dos parâmetros

```
# vectorization should also applied here
for j in range(n):
    w[j] = w[j] - alpha * dw[j]
b = b - alpha * db
```

- Evitar *loops*
- CPU e GPU possuem paralelização (SIMD)
- Colocamos cada exemplo x em uma coluna de uma matriz X

```
Z = np.dot(w.T, X) + b
A = sigmoid(Z)
dz = A - Y

dw = 1/m * np.dot(X, dz.T)
db = 1/m * np.sum(dz)

w = w - alpha * dw
b = b - alpha * db
```

Calorias por fonte:

	Apples	Beef	Eggs	Potatoes
Carb	56.0	0.0	4.4	68.0
Protein	1.2	104.0	52.0	8.0
Fat	1.8	135.0	99.0	0.9

Para cada alimento, obter a porcentagem de caloria advinda de cada fonte:

```
cal = A.sum(axis=0)  
perc = 100*A/cal.reshape(1,4)
```

Calorias por fonte:

	Apples	Beef	Eggs	Potatoes
Carb	56.0	0.0	4.4	68.0
Protein	1.2	104.0	52.0	8.0
Fat	1.8	135.0	99.0	0.9

Para cada alimento, obter a porcentagem de caloria advinda de cada fonte:

```
cal = A.sum(axis=0)  
perc = 100*A/cal.reshape(1,4)
```

Broadcasting - Exemplo

$$\begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix} + \begin{bmatrix} 100 \\ 100 \\ 100 \\ 100 \end{bmatrix} \quad \text{100}$$

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \quad \begin{matrix} \swarrow \\ (m,n) \end{matrix} \quad \begin{matrix} (2,3) \end{matrix} + \begin{bmatrix} 100 & 200 & 300 \\ 100 & 200 & 300 \end{bmatrix} \quad \begin{matrix} (1,n) \rightsquigarrow (m,n) \\ (2,3) \end{matrix}$$

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \quad \begin{matrix} (m,n) \end{matrix} + \begin{bmatrix} 100 & 100 & 100 \\ 200 & 200 & 200 \end{bmatrix} \quad \begin{matrix} (m,1) \\ \downarrow \\ (m,n) \end{matrix}$$

Broadcasting - Geral

$$\begin{array}{ccc} (m, n) & + & (1, n) \rightsquigarrow (m, n) \\ \text{matrix} & * & \\ \hline & / & (m, 1) \rightsquigarrow (m, n) \end{array}$$

$$\begin{array}{ccccc} (m, 1) & + & \mathbb{R} & & \\ \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} & + & 100 & = & \begin{bmatrix} 101 \\ 102 \\ 103 \end{bmatrix} \\ [1 \ 2 \ 3] & + & 100 & = & [101 \ 102 \ 103] \end{array}$$

Matlab/Octave: bsxfun

Rank 1 Array

Evite:

```
a = np.random.randn(5)
a.shape
# (5,)
```

Em vez disso, use:

```
a = np.random.randn(5,1)
a = np.random.randn(1,5)
```

Use *reshape* sempre que necessário:

```
a = a.reshape(5,1)
a.shape
# (5,1)
```